GRAPH_ANOMALY

FEW-SHOT TEMPORAL ANOMALY GENERATION AND DETECTION IN BRAIN CONNECTIVITY SUBGRAPHS FOR AUTISM DISORDER

End-to-End Technical Walkthrough — Phase 1 through Phase 3

What was built, why each step was taken, and the numerical evidence for what it actually achieved.

Dataset: ABIDE I · 871 subjects · 20 sites · 360-ROI parcellation

Author: Harsh Patel · Prepared: 28 September 2026

0. How This Project Is Structured

This document walks through **every stage** of GRAPH_ANOMALY in the order it actually runs, end to end: raw fMRI scans in, a trained detector and generator out. For every stage it answers three questions — **what** the step does, **why** that specific design was chosen over the obvious alternatives, and **what number came out** when it was actually run. Every metric in this document is copied from a committed results file in the repository (`outputs/experiments/phase{1,2,3}/*.json`) — nothing here is estimated or rounded from memory.

The project has three phases, and they build on each other strictly in order:

- **Phase 1** teaches a model what *normal* brain-subgraph behavior over time looks like, and uses deviation from that norm as a baseline anomaly detector.
- **Phase 2** is the project's main contribution: it uses that same trained model to *deliberately manufacture* realistic abnormal subgraph transitions, and proves those synthetic anomalies are meaningfully different from random noise.
- **Phase 3** asks the payoff question — if you only have a handful of real abnormal examples, does adding Phase 2's synthetic ones actually make a detector better?

> **Why this ordering, not classification-first**
> A direct 'ASD vs. control' image classifier was the obvious first idea and was deliberately not built. ABIDE gives a subject-level diagnosis label, not a label on any specific brain-connectivity event — so a classifier would be learning to associate a diagnosis with whatever incidental patterns correlate with site, scanner, or age, not with a specific abnormal mechanism. Framing the problem as *deviation from learned normal dynamics* instead gives a score that is at least mechanistically interpretable: it says exactly which subgraph, at which moment, deviated from what healthy subjects do — even before asking whether that deviation correlates with diagnosis.

1. The Data: ABIDE I

What. The project uses ABIDE I (Autism Brain Imaging Data Exchange), a public, multi-site resting-state fMRI dataset. Every subject contributes a functional MRI recording, already reduced by the dataset's own pipeline to one time series per brain region (ROI).

Quantity	Value
Total subjects	871
Diagnosis split	403 ASD / 468 typical control
Acquisition sites	20 (different scanners, protocols, populations)
Brain regions (ROIs)	360 per subject
Scan length	Variable, 78–316 time points (TRs) per subject
Train / Val / Test split	609 (328C/281A) / 131 (70C/61A) / 131 (70C/61A)
Split strategy	Subject-level, stratified by diagnosis, seed = 42

Why subject-level, not window-level, splitting
Splitting by time-window instead of by subject would let two windows from the *same* subject's scan land in both train and test — the model could partly memorize that subject's individual connectivity fingerprint rather than learning generalizable dynamics, silently inflating every downstream number. Every phase re-checks this: Phase 3's code explicitly asserts that support subjects and generation-base subjects are disjoint from the validation and test pools before every run (`assert_subject_isolation()`).

---

2. Step 1 — Turning a Scan Into a Sequence of Graphs

What. Each subject's 360-ROI time series is cut into overlapping time windows, and inside each window a 360×360 functional-connectivity (FC) matrix is computed — the Pearson correlation between every pair of ROIs' activity in that window. That matrix is then sparsified down to its strongest 20% of connections (kept *signed*, so an inhibitory / anti-correlated connection is preserved, not just its magnitude) and treated as a graph: 360 nodes (ROIs), weighted edges (connectivity strength). One subject therefore becomes not one graph, but a *sequence* of graphs, one per time window.

Parameter	Value	Meaning
Window size	30 TR	How many time points form one snapshot
Stride	10 TR	How far the window slides between snapshots
Connectivity metric	Pearson r, signed	Correlation between two ROIs' time series
Sparsification	Top 20% by |r|	Keeps only the strongest connections per window
Self-loops / diagonal	Removed	A region isn't connected to itself

Why a sliding window instead of one graph per subject
A single graph per subject (correlating the whole scan at once) throws away exactly the thing this project needs: whether connectivity *changes* over time. A 30-TR window with 10-TR stride is a standard dynamic-functional-connectivity setup — short enough to catch a state change, long enough for a stable correlation estimate from a noisy BOLD signal. Sparsifying to the top 20% removes weak, likely-spurious correlations that would otherwise dominate the parameter count of every downstream graph model.

---

3. Step 2 — From a 360-Node Graph to a Trackable Subgraph

What. A full 360-ROI graph per time window, per subject, across hundreds of subjects, is too large to train a model on directly and too large to interpret ('the brain changed' is not an actionable finding). So the project picks a **seed ROI** and extracts its 1-hop neighborhood — that ROI plus everything directly connected to it — as a much smaller subgraph, and does this consistently across every time window for that subject.

Stable vs. dynamic tracking. Two ways to define 'the same subgraph over time' were tried: let the neighborhood membership change window-to-window as connectivity shifts (*dynamic*), or fix the ROI membership once at a reference window and only let internal edge weights evolve (*stable*). Measured directly: dynamic tracking gave a

mean window-to-window Jaccard similarity of only **0.378** — the subgraph's membership was barely the same graph from one window to the next, making 'has this subgraph changed' impossible to answer cleanly. Stable tracking was adopted instead.

> **Why fixed membership, not evolving membership**
> If the set of ROIs in 'the subgraph' is allowed to change between window t and window t+1, then a large anomaly score could simply mean 'the neighborhood definition changed,' not 'this specific brain circuit behaved abnormally.' Fixing the ROI set at a reference window (t=0) means every later window is describing *the same physical circuit*, so a change in the subgraph's connectivity strength has one, unambiguous, interpretable meaning.

Which seeds. Evaluating all 360 possible seed ROIs across all subjects, every epoch, was measured at ~71 seconds per subject just for extraction — roughly 12 hours per pass over the 609 training subjects, repeated every training epoch. This is a laptop-scale project, so the project fixes a deterministic, stratified subsample of **20 seed ROIs** (chosen once with `selection_seed=42`), cached to disk. This is documented explicitly as a compute-budget decision, not a limitation of the extraction code itself, which supports all 360 ROIs as seeds.

---

4. Step 3 — Turning a Subgraph Into a Number Vector (GIN)

What. A subgraph — a set of nodes and weighted edges — is not something a GRU or a classifier can consume directly. A **Graph Isomorphism Network (GIN)** is used to compress each subgraph snapshot into a single 128-dimensional embedding vector: two message-passing layers (hidden width 64) let each node's representation absorb information from its neighbors, weighted by the real signed connectivity strength on each edge; a global mean-pool combines all node representations into one subgraph-level vector; a final linear layer projects that to 128-D.

GIN setting	Value
Node input feature	constant 1.0 (identity carried entirely by graph structure, not by pre-labeled node features)
Layers	2, hidden width 64
Message weighting	real signed edge weight from M1
Pooling	global mean pool over nodes
Output embedding	128-D per subgraph snapshot

> **Why GIN specifically, not a generic GCN**
> GIN was proven (Xu et al., ICLR 2019) to be as discriminative as the Weisfeiler-Lehman graph isomorphism test — the theoretical ceiling for message-passing networks — while common alternatives like plain GCN or GraphSAGE are provably weaker at telling structurally different graphs apart. Since this project's entire premise rests on detecting subtle structural differences between a normal and an abnormal subgraph, using the most structurally discriminative encoder available was a deliberate, not default, choice.

---

5. Phase 1 — Learning What 'Normal' Looks Like, and a Baseline Detector

What happens

The GIN encoder above and a GRU (Gated Recurrent Unit) are trained **jointly, end-to-end, from scratch** — the GIN is not pretrained separately. For every subgraph sequence $z_1, z_2, \ldots, z_T$ (one embedding per time window), the GRU's only job is to predict the next embedding $\hat{z}_{T+1}$ before being shown it. Critically, this is trained **only on the 328 train-control subjects × 20 seed ROIs** — no ASD subjects, and no validation or test subjects, enter training at all. The model therefore only ever learns what a *healthy* brain subgraph does over time.

> **Why predict-the-next-embedding, not a direct classifier**
> Training only on controls and scoring by *prediction error* means the model never needs an ASD label to learn from — it only needs to get good at predicting healthy dynamics. Anything that then deviates from what it learned to predict, ASD-related or not, shows up as a high error. This sidesteps the earlier stated problem of a diagnosis label being too coarse a supervisory signal for a transition-level question.

The anomaly score and the calibration bug that was caught

The anomaly score is simply the distance between the real next embedding and the GRU's predicted one: $A_t = ||z(t+1) - \hat{z}(t+1)||$. A threshold is then calibrated on the score distribution of *validation-control* subjects (95th percentile) so that a subject is flagged only if their worst moment is more extreme than 95% of healthy validation subjects' worst moments.

> **A real bug, found by running the numbers, not assumed away**
> The first calibration approach used one threshold per *individual transition*, flagging a subject if **any** of its roughly 280 transitions crossed it. Result: **131 of 131** test subjects were flagged — both healthy and ASD, indiscriminately. The cause: with ~280 independent rolls of the dice at a 95th-percentile cutoff, about 14 are expected to cross the line *by chance alone* for any subject, healthy or not (a multiple-comparisons problem). The fix was a second threshold, $\tau_{subject}$, calibrated directly on each subject's own worst-transition statistic rather than on any single transition — which is what the detector actually uses to decide.

Numbers — what came out

Quantity	Value	Note
Training epochs	5	epoch 4 checkpointed (best val loss), not the final epoch
Best validation loss	0.113	next-embedding prediction MSE
Post-training embedding std	1.03 – 1.16	collapse would show as std ≈ 0
τ_transition (P95, val-control)	7.435	per-window flag line
τ_subject (P95, val-control)	19.813	per-subject flag line, actually used

Quantity	Value	Note
Test Precision	0.500	2 of 4 flagged subjects were truly ASD
Test Recall	0.033	2 of 61 ASD subjects were caught
Test F1	0.062	—
Test AUROC	0.453	0.500 is a coin flip
Test AUPRC	0.467	baseline (class) rate is 0.466
Subjects flagged, out of 131 test	4	—

Read honestly
AUROC 0.453 is at chance. This is reported as measured, not hidden or reframed — ABIDE gives only a whole-subject diagnosis label, not a label on any specific transition, so a subject-level 'worst moment' score is a genuinely weak proxy for a transition-level phenomenon. Phase 1's honest role in this project is to be the **floor**: the reference point Phase 2 and Phase 3 are measured against, not a headline result.

What is genuinely positive here, numerically: the embedding standard deviation of 1.03–1.16 confirms the model did *not* take the trivial shortcut of collapsing every subgraph to the same output vector (which would read as std ≈ 0 and would make the loss look good while learning nothing) — a real, checked failure mode that did not occur. And the two-threshold fix took the false-flag rate from 131/131 (100%, useless) down to a believable 4/131 (3.1%), an outcome directly attributable to catching and fixing the multiple-comparisons bug above.

---

6. Phase 2 — Manufacturing Meaningful Anomalies (Main Contribution)

What happens

Phase 2 takes a real transition from a real, *normal* (train-control) subject and deliberately perturbs it into an anomaly — but not by adding random noise. It uses the Phase 1 model's own gradient: for a real transition, it computes how much each edge weight would need to change to *increase* the model's own prediction error the most (`∂(error)/∂(edge weight)`), and pushes exactly those edges. Concretely: the top 15% of undirected edge-pairs by combined sensitivity are selected, and 5 small gradient-ascent steps (recomputing the gradient each time) are taken, clipped to stay in the valid connectivity range and kept symmetric (`w_ij = w_ji`). Node/edge *identity* is never changed — only existing edge weights move, so the result is still a structurally valid brain subgraph, just one whose connectivity strengths are now abnormal in a model-relevant way.

Why gradient-guided perturbation, not a generative model (GAN/VAE)
A GAN or VAE would need to learn the entire distribution of 'valid subgraph' from scratch and would risk producing structurally implausible graphs, exactly the over-engineering this project explicitly avoids given the dataset size and laptop compute budget. Gradient-guided perturbation instead reuses the model that Phase 1 already trained and already validated (no representation collapse) — it asks that same model 'which change would surprise you the most', which is both simpler to implement correctly and directly interpretable: the generated anomaly is anomalous specifically *according to the model whose job is to detect anomalies*, not according to an unrelated generator's separate notion of plausibility.

A real design bug, found and fixed during development

One big step vs. five small steps
The first implementation took one large gradient step directly proportional to severity. Testing severities {0.25, 0.55, 0.9} on a held-out example gave anomaly scores **5.28** → **8.03** → **5.03** — not monotonic: a large step overshoots the region where the gradient's straight-line direction stays valid, because GIN's ReLU nonlinearities mean a big jump can cross into a different local regime entirely (the same overshoot failure mode known from single-step FGSM-style adversarial attacks at large budgets). Switching to 5 small, recomputed gradient steps (a PGD-style approach) fixed this: re-tested on 5 subjects × 8 severities, the structure-aware score became strictly increasing every time (`5.76→6.04→6.84→7.54→8.17→8.67→9.38→9.76`), while a random-perturbation baseline of the identical edit budget stayed flat and noisy (`5.76→5.75→5.79→5.67→5.77→5.40→6.17→6.03`).

Numbers — what came out (7,200 generated records, validated at scale)

Severity (α)	Structure-aware score (mean ± std)	Random score (mean ± std)
Low (0.25)	4.02 ± 2.45	2.91 ± 2.35
Medium (0.55)	5.82 ± 2.69	2.88 ± 2.35
High (0.90)	7.63 ± 3.17	2.84 ± 2.31
Monotonic in severity?	Yes	No

Source of transition	Fraction exceeding τ_transition (7.435)
Real normal (val-control)	0.050
Real ASD (test)	0.049
Generated, random, high severity	0.048
Generated, structure-aware, low severity	0.078
Generated, structure-aware, medium severity	0.230
Generated, structure-aware, high severity	0.458

The headline Phase 2 number: at high severity, structure-aware anomalies are flagged by Phase 1's own detector **45.8%** of the time, versus **4.8%** for a random perturbation of the exact same edit budget — roughly a **9×** difference,

and it required no threshold tuning to appear. Validity was perfect: **0 constraint violations across all 7,200 generated records** (correct range, symmetry, no self-loops). Diversity was near-maximal: mean pairwise Jaccard distance between different generated anomalies' touched-edge sets was **0.999–1.000** in every severity/method group, meaning the generator essentially never produces near-duplicate anomalies.

> **The one sobering number in Phase 2**
> Real ASD test transitions are detected at essentially the *same* rate as real normal transitions (0.049 vs. 0.050) — consistent with Phase 1's near-chance AUROC. Real clinical differences are far subtler than even a severe synthetic perturbation at this threshold. This is exactly the open question Phase 3 investigates next.

---

7. Phase 3 — Does the Generated Data Actually Help?

What happens

Phase 3 is the payoff experiment. The trained Phase 1 GIN+GRU is **frozen** (it is not retrained). A very small, simple classifier — logistic regression, deliberately not a second deep network — is fit on top of a 20-dimensional per-seed anomaly-score feature vector (one dimension per tracked seed ROI). Three conditions are compared, on the identical held-out test set (131 subjects), for identical support-set draws:

- **A — few-shot only:** *k* real ASD support subjects as positives vs. all 328 train-control subjects as negatives.
- **B — few-shot + structure-aware:** A's positives, plus 30 Phase-2 structure-aware generated anomalies (high severity).
- **C — few-shot + random (ablation):** A's positives, plus 30 random-perturbation generated anomalies.

$k \in \{1, 5, 10\}$, with 5 independent support-set draws per shot count (45 classifiers fit and evaluated in total).

A second real design bug, found and fixed

> **A one-number feature made the result meaningless before this was caught**
> The first version scored each subject with Phase 1's single scalar (max error across everything). A 1-D logistic regression is a strictly monotonic function of one number, so no matter how the classifier is trained, its *ranking* of test subjects is identical to just sorting by the raw feature — meaning AUROC/AUPRC came out **bit-for-bit identical** across all three conditions at every shot count. This was caught by actually inspecting the numbers, not assumed safe. The fix: a 20-dimensional per-seed feature vector, whose *learned weights* (not just a decision threshold) can genuinely change with the training data.

Numbers — the full required comparison

Shots	Condition	Precision	Recall	F1	AUROC	AUPRC
1	few-shot only	0.200	0.003	0.006	0.516	0.489
1	+ structure-aware	0.443	0.344	0.387	0.497	0.470
1	+ random	0.473	0.357	0.403	0.506	0.487

Shots	Condition	Precision	Recall	F1	AUROC	AUPRC
5	few-shot only	0.430	0.092	0.149	0.485	0.466
5	+ structure-aware	0.447	0.348	0.390	0.486	0.474
5	+ random	0.451	0.361	0.399	0.502	0.483
10	few-shot only	0.433	0.249	0.311	0.524	0.486
10	+ structure-aware	0.445	0.367	0.402	0.501	0.481
10	+ random	0.474	0.400	0.431	0.515	0.473

The clear win: at 1-shot, few-shot-only nearly collapses (F1 = 0.006 — a single positive example gives logistic regression almost nothing to calibrate a decision boundary from). Adding 30 generated examples of *either* kind restores F1 to roughly **0.39–0.40** — a ~65× improvement in F1, and recall goes from 0.003 (essentially never finding a real ASD subject) to roughly 0.35 (finding about one in three). This pattern holds, smaller in magnitude, at 5-shot and 10-shot too. Error bars across the 5 trials are tight relative to this gap — it is a real, reproducible effect of having more labeled training data, not noise.

The honest, important nuance

AUROC/AUPRC stay near chance (0.48–0.53) in **every** condition, including the generated-data ones — the F1/Recall gain comes from giving the classifier a usable decision boundary, not from making the underlying feature space more separable. And structure-aware generation does **not** show a clear edge over random generation in this classifier's ranking metrics, despite Phase 2's own validation showing structure-aware anomalies are detected ~9× more often than random by the raw per-transition threshold. A plausible, unconfirmed explanation given in the project's own report: each generated example only edits 1 of 20 feature dimensions, and with just 30 generated examples spread across up to 20 seed identities per trial, there may be too little per-dimension signal for a *linear* classifier to learn *which* seed's anomaly matters — so it mostly benefits from 'more positive-labeled data exists' rather than from the structure-aware signal specifically. No thresholds or support sets were adjusted after seeing this result to make it look better; it is reported as the first and only run.

8. The Positive Results, Numerically

Set aside the near-chance AUROC for a moment — that number alone undersells what actually worked in this project. Here is every concrete, numerically-verified positive result, in one place:

1. **No representation collapse.** Post-training embedding std stayed at 1.03–1.16 across dimensions (collapse would read ≈ 0). The GIN did not take the shortcut of mapping every subgraph to the same vector to minimize loss.
2. **A real detection bug was caught and fixed before any number was reported.** A single per-transition threshold flagged 131 of 131 test subjects (100%, a multiple-comparisons failure). Recalibrating on the subject-level statistic brought this to a believable 4 of 131 (3.1%).
3. **Structure-aware perturbation is strictly severity-monotonic; random is not.** Mean anomaly score rises 4.02 → 5.82 → 7.63 as severity increases for structure-aware generation (a clean, controllable dial). The random baseline stays flat at 2.84–2.91 regardless of severity.
4. **Structure-aware anomalies are ~9× more detectable than random, at an identical edit budget.** 45.8% detection rate for structure-aware at high severity vs. 4.8% for random — the central, validated claim that the generator is exploiting real, model-relevant structure, not just injecting bigger numbers.
5. **Zero constraint violations across 7,200 generated anomalies.** Every generated record stayed within the valid connectivity range, kept edge symmetry, and added no self-loops — validated at scale, not spot-checked.
6. **Generated anomalies are diverse, not near-duplicates.** Mean pairwise Jaccard distance between touched-edge sets was 0.999–1.000 across every severity/method group.
7. **Adding generated data turns a collapsed 1-shot detector into a usable one.** F1 goes from 0.006 (few-shot-only, 1 real example) to 0.39–0.40 once 30 generated anomalies are added — recall from 0.003 to roughly 0.35–0.36, a reproducible effect with tight error bars across 5 independent trials.
8. **Two independent classifier-design bugs were caught by actually running the numbers, not assumed away.** A rank-invariant 1-D feature (Phase 3) and a multiple-comparisons threshold bug (Phase 1) both produced suspicious, too-good-or-too-uniform results that were investigated and fixed rather than reported as-is.

9. What Is Still Genuinely Unresolved

Stated as plainly as the wins above, because the project's own reports insist on this:

- **Phase 1's baseline AUROC (0.453) is at chance** — ABIDE's subject-level diagnosis label is a weak stand-in for a transition-level anomaly label, and this is a real ceiling on everything built on top of it.

- **Structure-aware generation's advantage does not clearly transfer into Phase 3's classifier metrics** (AUROC/AUPRC) even though it clearly exists in Phase 2's own raw detectability check — the most important open gap in the project.
- **Only 20 of 360 candidate ROIs are used as seeds**, a documented compute-budget subsample, not a capability limit of the extraction code.
- **The generator only edits existing edge weights**, never adding/removing ROIs or edges — so 'structural' anomalies here are connectivity-*strength* anomalies, not topology-changing ones.
- **Real ASD transitions are not well separated from real normal ones** by this baseline detector at all (0.049 vs. 0.050 detection rate) — even severe synthetic perturbation is more obviously abnormal than real clinical signal is, at this threshold.
- No claim of clinical diagnosis or biological causal mechanism is made anywhere in this project; it is a research prototype for anomaly generation and detection methodology.

---

10. Where To Find the Source of Every Number

Phase	Config	Results file	Full report
1	configs/phase1.yaml	outputs/experiments/phase1/metrics.json	docs/phase1_report.md
2	configs/phase2.yaml	outputs/experiments/phase2/generator_validation.json	docs/phase2_report.md
3	configs/phase3.yaml	outputs/experiments/phase3/final_comparison.json	docs/phase3_report.md

Every stage is seeded (seed=42 throughout) and config-driven, so every number in this document can be regenerated exactly by re-running the corresponding `python -m src.…` *command listed in* `docs/codebase_guide.md`.